

A dispatch queue is made up of two things: a linked list of waiting tasks, and a pool of worker threads. Worker threads are created once when the queue is created and remain sleeping until a task arrives. The main thread adds tasks to the linked list and wakes workers up. Dispatching refers to the process where a worker thread removes a task from the queue's linked list and executes it. This happens independently of how tasks are submitted (sync or async).

The key difference between serial and concurrent queues is how many worker threads exist, for serial it's one, for concurrent it's the same number as the number of cores present.

We first create the queue struct which holds everything needed to coordinate thread workers and tasks: (I've added comments to explain each part's purpose in my code)

```
struct dispatch_queue_t {
    queue_type_t queue_type; // SERIAL or CONCURRENT
    dispatch_queue_thread_t *threads; // the worker threads
    int thread_count; // 1 for SERIAL, num cores for CONCURRENT
    task_node_t *head; // front of linked list (next to run)
    task_node_t *tail; // back of linked list (newest task)
    pthread_mutex_t mutex; // protects linked list from corruption
    pthread_cond_t all_done; // what dispatch_queue_wait sleeps on
    sem_t task_semaphore; // workers sleep on this, posted when task added
    int running; // 1 = alive, 0 = shutting down
    int accepting_tasks; // 1 = taking tasks, 0 = after wait called
    int tasks_in_progress; // how many workers are currently running a task
};
```

Then we create the dispatch queue using `dispatch_queue_create` for setting it up:

```
queue->queue_type = queueType;
queue->head = NULL; //sets up linked list
queue->tail = NULL; //sets up linked list
queue->running = 1;
queue->accepting_tasks = 1;
queue->tasks_in_progress = 0;

if (pthread_mutex_init(&queue->mutex, NULL) != 0) //sets up mutex
    error_exit("pthread_mutex_init failed");
if (pthread_cond_init(&queue->all_done, NULL) != 0) //sets up condition variable
    error_exit("pthread_cond_init failed");
if (sem_init(&queue->task_semaphore, 0, 0) != 0) //sets up semaphore, 0 for shared between threads, 0 for initial value
    error_exit("sem_init failed");
```

Then it creates the worker threads:

```
queue->thread_count = (queueType == SERIAL) ? 1
    : (int)sysconf(_SC_NPROCESSORS_ONLN);
```

SERIAL gets exactly 1 worker thread. CONCURRENT gets one per CPU core. Then after the setup, the threads are created.

```
for (int i = 0; i < queue->thread_count; i++) { //after set up threads are created
    queue->threads[i].queue = queue;
    if (pthread_create(&queue->threads[i].thread, NULL,
        worker_thread, &queue->threads[i]) != 0)
        error_exit("pthread_create failed");
}
```

Queue is then returned.

Worker threads start immediately, every worker thread sits in an infinite loop blocked on the queue semaphore (`sem_wait`) which starts at 0, awaiting tasks to arrive:

```

void *worker_thread(void *arg) {
    dispatch_queue_thread_t *thread = (dispatch_queue_thread_t *)arg;
    dispatch_queue_t *queue = thread->queue;

    while (1) {
        sem_wait(&queue->task_semaphore); // sleep here until a task arrives

        pthread_mutex_lock(&queue->mutex);

        if (!queue->running) { // check if shutting down
            pthread_mutex_unlock(&queue->mutex);
            break; //exit loop
        }

        if (queue->head == NULL) {
            pthread_mutex_unlock(&queue->mutex);
            continue;
        }
        // dequeue task from front of linked list (FIFO)
        task_node_t *node = queue->head;
        queue->head = node->next;
        if (queue->head == NULL)
            queue->tail = NULL;

        queue->tasks_in_progress++;
        pthread_mutex_unlock(&queue->mutex);

        task_t *task = node->task;
        free(node);

        task->work(task->params); //run the actual function...
    }
}

```

The mutex protects the linked list so only one worker dequeues at a time. After dequeuing, the worker threads release the mutex and run the task independently.

Both `dispatch_async` and `dispatch_sync` use a shared helper to add tasks:

```

static void enqueue_locked(dispatch_queue_t *queue, task_node_t *node) {
    node->next = NULL;
    if (queue->tail == NULL) {
        queue->head = node;
        queue->tail = node;
    } else {
        queue->tail->next = node;
        queue->tail = node;
    }
}

```

New tasks always go to the tail. Workers always take from the head. This guarantees strict FIFO order.

Tasks can be added two ways regardless of queue type (serial or concurrent). Both ways use the same enqueue helper to add to the linked list - the difference is what happens after (main thread waiting or not), which is controlled by two completely separate semaphores:

- **queue->task_semaphore** - shared by everyone, lives on the queue. Every time any task is added this gets incremented by 1. Workers all watch this same semaphore. When it goes above 0, one worker wakes up. It is essentially a counter of how many tasks are waiting to be picked up.
- **task->task_semaphore** - personal to one specific sync task, created fresh each time `dispatch_sync` is called. Only two threads ever touch it - the main thread that submitted the task waits on it, and the worker that runs the task posts to it when done. It is a private signal between exactly those two threads.

The queue->task_semaphore is what directly triggers dispatching - when a task is added sem_post increments the semaphore - if a worker is sleeping it wakes up immediately, if all workers are busy the counter stays incremented until one becomes free and loops back.

With these two semaphores in mind, here is how each dispatch method uses them in my implementation:

1. ASYNC (dispatch_async) - main thread adds the task and returns immediately - it doesn't wait:

```
task->type = ASYNC;

enqueue_locked(queue, node); // add to tail of linked list
pthread_mutex_unlock(&queue->mutex);

sem_post(&queue->task_semaphore); // ring bell to wake a worker
// main is gone, does not wait at all
```

2. SYNC (dispatch_sync) - main adds the task then freezes until it is done:

```
task->type = SYNC;
if (sem_init(&task->task_semaphore, 0, 0) != 0) // create a personal bell for this task
    error_exit("task semaphore init failed");

enqueue_locked(queue, node); // add to tail of linked list
pthread_mutex_unlock(&queue->mutex);

sem_post(&queue->task_semaphore); // ring bell to wake a worker

/* Block until the worker posts after executing the task */
sem_wait(&task->task_semaphore); // main freezes here until worker done

/* Caller owns cleanup now that task is finished */
task_destroy(task); // main wakes up and cleans up
```

After the worker finishes running the task it checks the type:

For SYNC tasks, ownership of the task memory transfers to main. The worker never touches the task again after posting the semaphore. For ASYNC tasks the worker does 100% of the cleanup itself since nobody is waiting.

```
if (task->type == SYNC) {
    sem_post(&task->task_semaphore); // ring personal bell to wake main
    // worker walks away, main owns cleanup
    /* caller (dispatch_sync) owns task_destroy */
} else {
    task_destroy(task); // async: nobody waiting, worker cleans up itself
}

pthread_mutex_lock(&queue->mutex);
queue->tasks_in_progress--;
if (queue->tasks_in_progress == 0 && queue->head == NULL)
    pthread_cond_broadcast(&queue->all_done); // signal dispatch_queue_wait
pthread_mutex_unlock(&queue->mutex);
```

Dispatching a serial queue:

Dispatching means selecting the next task from the head of the FIFO linked list, incrementing tasks_in_progress, releasing the mutex, then executing the task function.

```
queue->thread_count = (queueType == SERIAL) ? 1
: (int)sysconf(_SC_NPROCESSORS_ONLN);
```

In my implementation, a SERIAL queue has exactly 1 worker thread. Because there is only one worker, it can only run one task at a time. It finishes task A completely, loops back, picks up task B, finishes it, and so on. Tasks always run in strict FIFO order. There is no purpose for serial queues to have more than one thread because tasks must run one after another by definition, so extra threads would just sit idle and unused; one thread is sufficient.

This is why in test5:

- Tasks always print in order A through J
- The counter always reaches exactly 10,000,000,000
- Only one CPU core is ever busy at a time

There is no possibility of two tasks running simultaneously because there is physically only one thread to run them.

Dispatching a concurrent queue:

Also works using FIFO from the head of the linked list. The difference is multiple workers can dispatch simultaneously, each grabbing the next available task.

In my implementation, a CONCURRENT queue creates one worker thread per CPU core (so for me with 4 cores, 4 threads):

```
queue->thread_count = (queueType == SERIAL) ? 1
: (int)sysconf(_SC_NPROCESSORS_ONLN);
```

All workers sleep on the same queue semaphore. When tasks are added they all race to dequeue. The mutex ensures only one dequeue at a time, but after dequeuing they run completely in parallel on separate cores. This is why in test4:

- All CPU cores hit 100% simultaneously
- Tasks print in unpredictable order
- The counter never reaches 10,000,000,000 because multiple threads write to it at the same time without synchronisation, overwriting each other's work

In summary, serial dispatches one task at a time through a single worker, concurrent dispatches multiple tasks simultaneously through N workers. Both use the same FIFO linked list and worker loop - the only difference is how many workers are consuming from it.